Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (Bsc.IT Semester III)
Data Structures Practical

Practical - V

Roll No. : S009	Name: Shubham Gawde
Class : SYIT	Batch : 1
Date of Assignment : 22/07/26	Date/Time of Submission : 22/7/26

Binary Search Tree: Write a program to:

a. Create a binary search tree.

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')
nodeC = TreeNode('C')
nodeD = TreeNode('D')
nodeE = TreeNode('E')
nodeF = TreeNode('F')
nodeG = TreeNode('G')

root.left = nodeA
root.right = nodeB

nodeA.left = nodeC
nodeA.right = nodeD

nodeB.left = nodeE
nodeB.right = nodeF

nodeD.left = nodeG

print("root.right.left.data:", root.right.left.data)
print("S009")
```

```
ams/Python/Python314/DS5.py
root.right.left.data: E
S009
```

b. Insert nodes into a binary search tree.

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')
nodeC = TreeNode('C')
nodeD = TreeNode('D')
nodeE = TreeNode('E')
nodeF = TreeNode('F')
nodeG = TreeNode('G')
nodeI = TreeNode('I')

root.left = nodeA
root.right = nodeB

nodeA.left = nodeC
nodeA.right = nodeD

nodeB.left = nodeE
nodeB.right = nodeF

nodeD.left = nodeG

nodeF.right = nodeI

print("root.right.right.right.data:", root.right.right.right.data)
print("S009")
```

```
root.right.right.right.data: I
S009
```

c. Search for a node in a binary search tree.

class TreeNode:

def __init__(self, data):

self.data = data

self.left = None

self.right = None

```
root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')
nodeC = TreeNode('C')
nodeD = TreeNode('D')
nodeE = TreeNode('E')
nodeF = TreeNode('F')
nodeG = TreeNode('G')

root.left = nodeA
root.right = nodeB

nodeA.left = nodeC
nodeA.right = nodeD

nodeB.left = nodeE
nodeB.right = nodeF

nodeD.left = nodeG

def search(root,key):
    if root is None:
        return False
    print("Checking:",root.data)
    if root.data == key:
        return True
    return search(root.left,key) or search(root.right,key)

key = input("Enter element to search: ")
```

```
if search(root,key):
    print("Element Found")
else:
    print("Element not found")
print("S009")
```

```
= RESTART: C:/Users/mvluc/AppData/Local
Enter element to search: D
Checking: R
Checking: A
Checking: C
Checking: D
Element Found
S009

= RESTART: C:/Users/mvluc/AppData/Local
Enter element to search: S
Checking: R
Checking: A
Checking: C
Checking: D
Checking: G
Checking: B
Checking: E
Checking: F
Element not found
S009
```

Curiosity Questions:

1. In a BST, which side (left or right) has **smaller numbers** than the root?
2. If we insert numbers 10, 20, 5 into a BST, which number will be at the root?
3. What will the inorder traversal of a BST give — sorted numbers or random numbers?
4. Where will the **largest number** in a BST always be found?
5. If the root of a BST is 50 and we search for 70, will we go left or right? Why?
6. If a BST has only **one node**, what is its inorder traversal?
7. Can we create a BST from characters like A, B, C instead of numbers?
8. If we insert numbers 5, 10, 15, 20 in order, will the tree be balanced or like a straight line?
9. In a BST, do we check the left child first or the right child first in inorder traversal?
10. If we delete the root of a BST, will the tree disappear?